

Punto 4.7 - Consultas ineficientes: filtros no traducibles y funciones en Where

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren consultas ineficientes por filtros no traducibles y funciones en Where. Reescribir las consultas para que se traduzcan completamente a SQL. Comparar el SQL generado y el tiempo de ejecución.

Contexto del proyecto: En el punto 4.6 se estudió el over-fetching, identificando sus causas y aplicando proyecciones, filtros y paginación. En este punto se profundiza en las consultas ineficientes, que son aquellas que no se traducen completamente a SQL o que impiden el uso de índices. Esta técnica se usará en el punto 4.8 para las Split Queries.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de consultas ineficientes a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerPorClienteConFuncion(string cliente);
    List<OrdenFabricacion> ObtenerPorClienteSinFuncion(string cliente);
    List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado);
    List<OrdenFabricacion> ObtenerPorEstadoDirecto(string estado);
    string ObtenerSqlConFuncion(string cliente);
    string ObtenerSqlSinFuncion(string cliente);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerPorClienteConFuncion(string cliente); → método que usa ToLower en Where.

Línea 11: List ObtenerPorClienteSinFuncion(string cliente); → método que compara directamente.

Línea 12: List ObtenerPorMetodoPersonalizado(string estado); → método que usa un método personalizado.

Línea 13: `List ObtenerPorEstadoDirecto(string estado);` → método que compara directamente.

Línea 14: `string ObtenerSqlConFuncion(string cliente);` → método que devuelve el SQL con función.

Línea 15: `string ObtenerSqlSinFuncion(string cliente);` → método que devuelve el SQL sin función.

Línea 17: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 18: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

Paso 3: Implementar los métodos de consultas ineficientes en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<OrdenFabricacion> ObtenerPorClienteConFuncion(string cliente)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente.ToLower() == cliente.ToLower())
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorClienteSinFuncion(string cliente)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente == cliente)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado)
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    return ordenes.Where(o => EsEstadoValido(o.Estado, estado)).ToList();
}

public List<OrdenFabricacion> ObtenerPorEstadoDirecto(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Estado == estado)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public string ObtenerSqlConFuncion(string cliente)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente.ToLower() == cliente.ToLower());

    return consulta.ToQueryString();
}

public string ObtenerSqlSinFuncion(string cliente)
{
    var consulta = _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.Cliente == cliente);

    return consulta.ToQueryString();
}
```

```

    }

    private static bool EsEstadoValido(string estadoActual, string estadoBuscado)
    {
        return string.Equals(estadoActual, estadoBuscado, StringComparison.OrdinalIgnoreCase);
    }

```

Línea 1: public List ObtenerPorClienteConFuncion(string cliente) → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.Cliente.ToLower() == cliente.ToLower()) → filtra con ToLower sobre la columna.

Línea 6: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 7: .ToList(); → materializa la consulta.

Línea 10: public List ObtenerPorClienteSinFuncion(string cliente) → declara el método.

Línea 12: return _context.OrdenesFabricacion → inicia la consulta.

Línea 13: .AsNoTracking() → aplica AsNoTracking.

Línea 14: .Where(o => o.Cliente == cliente) → filtra directamente.

Línea 15: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 16: .ToList(); → materializa la consulta.

Línea 19: public List ObtenerPorMetodoPersonalizado(string estado) → declara el método.

Línea 21: var ordenes = _context.OrdenesFabricacion → inicia la consulta.

Línea 22: .AsNoTracking() → aplica AsNoTracking.

Línea 23: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 24: .ToList(); → materializa todas las órdenes en memoria.

Línea 26: return ordenes.Where(o => EsEstadoValido(o.Estado, estado)).ToList(); → filtra en memoria con un método personalizado.

Línea 29: public List ObtenerPorEstadoDirecto(string estado) → declara el método.

Línea 31: return _context.OrdenesFabricacion → inicia la consulta.

Línea 32: .AsNoTracking() → aplica AsNoTracking.

Línea 33: .Where(o => o.Estado == estado) → filtra directamente.

Línea 34: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 35: .ToList(); → materializa la consulta.

Línea 38: public string ObtenerSqlConFuncion(string cliente) → declara el método.

Línea 40: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 41: .AsNoTracking() → aplica AsNoTracking.

Línea 42: .Where(o => o.Cliente.ToLower() == cliente.ToLower()); → filtra con ToLower.

Línea 44: return consulta.ToQueryString(); → devuelve el SQL.

Línea 47: public string ObtenerSqlSinFuncion(string cliente) → declara el método.

Línea 49: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 50: .AsNoTracking() → aplica AsNoTracking.

Línea 51: `.Where(o => o.Cliente == cliente);` → filtra directamente.

Línea 53: `return consulta.ToString();` → devuelve el SQL.

Línea 56: `private static bool EsEstadoValido(string estadoActual, string estadoBuscado)` → declara el método personalizado.

Línea 58: `return string.Equals(estadoActual, estadoBuscado, StringComparison.OrdinalIgnoreCase);` → compara sin distinguir mayúsculas.

Error común: si el método personalizado se usa directamente en `Where` sobre `IQueryable`, EF Core lanza una excepción indicando que la expresión no se puede traducir. Se debe materializar antes o reescribir la consulta.

Paso 4: Crear el caso de uso de consultas ineficientes

Crear el archivo `src/AceriaData.Application/UseCases/ConsultasIneficientesUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ConsultasIneficientesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ConsultasIneficientesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CONSULTAS INEFICIENTES ===");

        DemostrarConFuncion();
        DemostrarSinFuncion();
        DemostrarMetodoPersonalizado();
        DemostrarEstadoDirecto();
        CompararSql();
    }

    private void DemostrarConFuncion()
    {
        Console.WriteLine("\n--- Consulta con función en Where ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorClienteConFuncion("Constructora del Norte");
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarSinFuncion()
    {
        Console.WriteLine("\n--- Consulta sin función en Where ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorClienteSinFuncion("Constructora del Norte");
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarMetodoPersonalizado()
    {
        Console.WriteLine("\n--- Consulta con método personalizado ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorMetodoPersonalizado("Pendiente");
    }
}
```

```

        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void DemostrarEstadoDirecto()
    {
        Console.WriteLine("\n--- Consulta con estado directo ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerPorEstadoDirecto("Pendiente");
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    }

    private void CompararSql()
    {
        Console.WriteLine("\n--- SQL con función ---");
        var sqlConFuncion = _unidad.Ordenes.ObtenerSqlConFuncion("Constructora del Norte");
        Console.WriteLine(sqlConFuncion);

        Console.WriteLine("\n--- SQL sin función ---");
        var sqlSinFuncion = _unidad.Ordenes.ObtenerSqlSinFuncion("Constructora del Norte");
        Console.WriteLine(sqlSinFuncion);
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ConsultasIneficientesUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ConsultasIneficientesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== CONSULTAS INEFICIENTES ==="); → muestra la cabecera.

Línea 19: DemostrarConFuncion(); → llama al método con función.

Línea 20: DemostrarSinFuncion(); → llama al método sin función.

Línea 21: DemostrarMetodoPersonalizado(); → llama al método con método personalizado.

Línea 22: DemostrarEstadoDirecto(); → llama al método con estado directo.

Línea 23: CompararSql(); → llama al método de comparación de SQL.

Línea 26: private void DemostrarConFuncion() → declara el método.

Línea 28: Console.WriteLine("\n--- Consulta con función en Where ---"); → muestra la cabecera.

Línea 30: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 31: var ordenes = _unidad.Ordenes.ObtenerPorClienteConFuncion("Constructora del Norte"); → llama al método del repositorio.

Línea 32: cronometro.Stop(); → detiene el cronómetro.

Línea 34: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra los resultados.

Línea 37: private void DemostrarSinFuncion() → declara el método.

Línea 39: `Console.WriteLine("\n--- Consulta sin función en Where ---");` → muestra la cabecera.

Línea 41: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 42: `var ordenes = _unidad Ordenes.ObtenerPorClienteSinFuncion("Constructora del Norte");` → llama al método del repositorio.

Línea 43: `cronometro.Stop();` → detiene el cronómetro.

Línea 45: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra los resultados.

Línea 48: `private void DemostrarMetodoPersonalizado()` → declara el método.

Línea 50: `Console.WriteLine("\n--- Consulta con método personalizado ---");` → muestra la cabecera.

Línea 52: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 53: `var ordenes = _unidad Ordenes.ObtenerPorMetodoPersonalizado("Pendiente");` → llama al método del repositorio.

Línea 54: `cronometro.Stop();` → detiene el cronómetro.

Línea 56: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra los resultados.

Línea 59: `private void DemostrarEstadoDirecto()` → declara el método.

Línea 61: `Console.WriteLine("\n--- Consulta con estado directo ---");` → muestra la cabecera.

Línea 63: `var cronometro = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 64: `var ordenes = _unidad Ordenes.ObtenerPorEstadoDirecto("Pendiente");` → llama al método del repositorio.

Línea 65: `cronometro.Stop();` → detiene el cronómetro.

Línea 67: `Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");` → muestra los resultados.

Línea 70: `private void CompararSql()` → declara el método.

Línea 72: `Console.WriteLine("\n--- SQL con función ---");` → muestra la cabecera.

Línea 73: `var sqlConFuncion = _unidad Ordenes.ObtenerSqlConFuncion("Constructora del Norte");` → obtiene el SQL.

Línea 74: `Console.WriteLine(sqlConFuncion);` → imprime el SQL.

Línea 76: `Console.WriteLine("\n--- SQL sin función ---");` → muestra la cabecera.

Línea 77: `var sqlSinFuncion = _unidad Ordenes.ObtenerSqlSinFuncion("Constructora del Norte");` → obtiene el SQL.

Línea 78: `Console.WriteLine(sqlSinFuncion);` → imprime el SQL.

Error común: si el método `EsEstadoValido` se usa directamente en `Where` sobre `IQueryable`, EF Core lanza una excepción. Se debe materializar antes.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ConsultasIneficientesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ConsultasIneficientesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

Paso 7: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context.OrganizacionFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de cada técnica y el SQL generado.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CONSULTAS INEFICIENTES ===

--- Consulta con función en Where ---
Órdenes: 2 | Tiempo: 48 ms

--- Consulta sin función en Where ---
Órdenes: 2 | Tiempo: 40 ms

--- Consulta con método personalizado ---
Órdenes: 4 | Tiempo: 55 ms

--- Consulta con estado directo ---
Órdenes: 4 | Tiempo: 41 ms

--- SQL con función ---
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE LOWER([o].[Cliente]) = @__ToLower_0 AND [o].[IsDeleted] = CAST(0 AS bit)

--- SQL sin función ---
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Cliente] = @__cliente_0 AND [o].[IsDeleted] = CAST(0 AS bit)
```

La primera sección muestra el tiempo con función. La segunda sección muestra el tiempo sin función. La tercera sección muestra el tiempo con método personalizado. La cuarta sección muestra el tiempo con estado directo. La quinta sección muestra el SQL con función. La sexta sección muestra el SQL sin función.

Observaciones: la consulta con función es más lenta porque SQL Server no puede usar el índice. La consulta con método personalizado es más lenta porque carga todas las órdenes en memoria y después filtra. El SQL con función incluye LOWER([o].[Cliente])). El SQL sin función compara directamente la columna.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerPorMetodoPersonalizado para usar el método personalizado directamente en Where:

```
public List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado)
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => EsEstadoValido(o.Estado, estado))
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado: EF Core lanza una excepción `InvalidOperationException` indicando que la expresión `EsEstadoValido` no se puede traducir a SQL.

Solución: materializar antes de aplicar el método personalizado o reescribir la consulta con una expresión traducible.

```
public List<OrdenFabricacion> ObtenerPorMetodoPersonalizado(string estado)
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    return ordenes.Where(o => EsEstadoValido(o.Estado, estado)).ToList();
}
```

Resultado esperado con la solución: el método funciona y filtra en memoria. La desventaja es que carga todas las órdenes en memoria.

Errores comunes del ejercicio

Error	Causa	Solución
Función en Where	Se aplica <code>ToLower</code> sobre la columna	Comparar directamente o usar <code>collation</code>
Método personalizado en Where	EF Core no puede traducirlo	Materializar antes o reescribir
Expresión regular en Where	EF Core no puede traducirla	Reescribir con <code>StartsWith</code> , <code>EndsWith</code> o <code>Contains</code>
Filtro en memoria	Se materializa antes de filtrar	Filtrar antes de materializar
Índice no usado	Se aplica una función sobre la columna	Evitar funciones en Where
Collation incorrecta	La <code>collation</code> es sensible a mayúsculas	Configurar <code>collation</code> insensible

Reto resuelto: Reescribir una consulta con expresión regular

Reto: Reescribir una consulta que usa una expresión regular para validar el formato del número de orden (OF-YYYY-NNNN) usando expresiones que EF Core pueda traducir a SQL.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz `IOrdenRepositorio`:

```
List<OrdenFabricacion> ObtenerPorFormatoNumeroOrden();
```

Línea 1: `List ObtenerPorFormatoNumeroOrden();` → declara el método.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion> ObtenerPorFormatoNumeroOrden()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Where(o => o.NumeroOrden.StartsWith("OF-") && o.NumeroOrden.Length == 12)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerPorFormatoNumeroOrden() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .Where(o => o.NumeroOrden.StartsWith("OF-") && o.NumeroOrden.Length == 12) → filtra por formato usando expresiones traducibles.

Línea 6: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 7: .ToList(); → materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarFormatoNumeroOrden()
{
    Console.WriteLine("\n--- Formato de número de orden ---");

    var ordenes = _unidad.Ordenes.ObtenerPorFormatoNumeroOrden();
    Console.WriteLine($"Órdenes con formato válido: {ordenes.Count}");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden}");
    }
}
```

Línea 1: private void DemostrarFormatoNumeroOrden() → declara el método.

Línea 3: Console.WriteLine("\n--- Formato de número de orden ---"); → muestra la cabecera.

Línea 5: var ordenes = _unidad.Ordenes.ObtenerPorFormatoNumeroOrden(); → llama al método del repositorio.

Línea 6: Console.WriteLine(\$"Órdenes con formato válido: {ordenes.Count}"); → muestra el número de órdenes.

Línea 7: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 9: Console.WriteLine(\$" {orden.NumeroOrden}"); → muestra el número.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarFormatoNumeroOrden();
```

Paso 5: Ejecutar dotnet run y verificar que las órdenes con formato válido se muestran correctamente.

Resultado esperado: las órdenes con formato OF-YYYY-NNNN se muestran correctamente. La consulta se traduce completamente a SQL usando LIKE 'OF-%' y LEN.

| Analogía final

Las consultas ineficientes en una acería son como pedir al archivo central que busque las carpetas cuyo nombre, al aplicarle una regla personalizada, coincida con un patrón. Si la regla es estándar, el archivo la entiende y busca directamente en el índice. Si la regla es personalizada, el archivo tiene que traer todas las carpetas y aplicar la regla él mismo, una por una. Las funciones en Where son como pedir que busque las carpetas cuyo nombre, convertido a minúsculas, coincida con un patrón. El archivo no puede usar el índice porque los nombres están en mayúsculas y minúsculas. Tiene que recorrer todas las carpetas. La solución es pedir que busque directamente por el nombre, sin conversiones, o configurar el archivo para que las comparaciones sean insensibles a mayúsculas. Los métodos personalizados son como pedir que busque las carpetas que cumplan una regla que solo el jefe conoce. El archivo no entiende la regla y tiene que traer todas las carpetas para que el jefe las revise. Así funciona la optimización de consultas en EF Core: se reescriben las expresiones no traducibles con expresiones que EF Core puede convertir a SQL, y se evitan las funciones sobre columnas para que los índices se usen.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de consultas ineficientes a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso ConsultasIneficientesUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones con función, sin función, con método personalizado y con estado directo.

Comparado el SQL generado con y sin función.

Diagnosticado el error de usar un método personalizado en Where.

Creado el método ObtenerPorFormatoNumeroOrden con expresiones traducibles.
